2022 É O ANO DA **realização** NO **GRUPO TIRADENTES**

GRUPO TIRADENTES

Inspirando o futuro

Introdução ao Python

PROF. DR. CARLOS GUSTAVO P. MORAES

2022 - ANO DA realização

GRUPO TIRADENTES
Inspirando o futuro

LINGUAGEM DE PROGRAMAÇÃO

A finalidade básica de um computador é realizar a tarefa de processamento de dados, ou seja, receber dados por um dispositivo de entrada (por exemplo, teclado, mouse, planilha, etc.), realizar operações com estes dados e gerar uma resposta que será expressa em um dispositivo de saída (monitor, impressora, etc.)

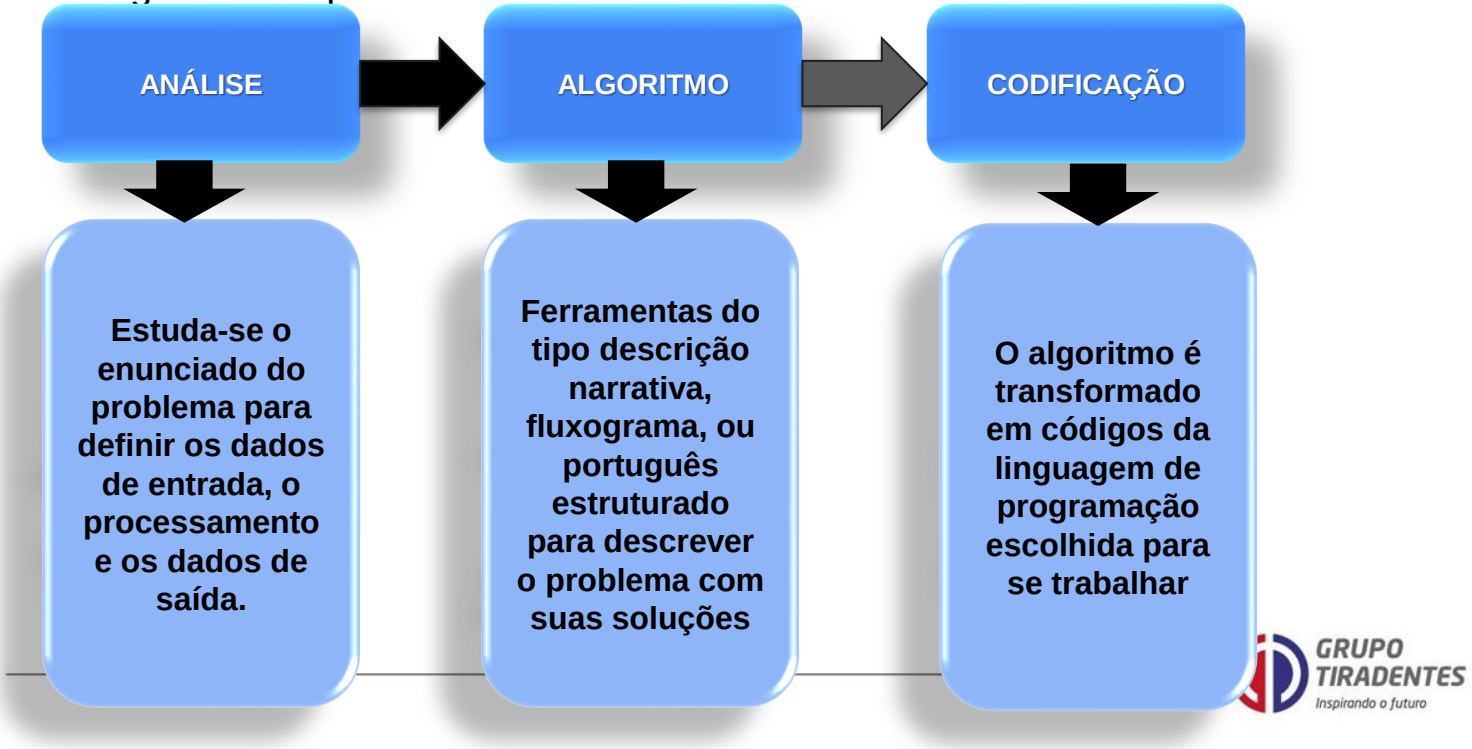
Quando desejamos criar ou desenvolver um software para realizar determinado tipo de processamento de dados, devemos escrever um programa ou vários programas interligados.

Para que o computador compreenda e execute este programa, devemos escrevê-lo utilizando uma linguagem que tanto o computador quanto o criador de software entendam.

E uma das principais etapas no desenvolvimento de um programa é o chamado **algoritmo**

LINGUAGEM DE PROGRAMAÇÃO

- Basicamente, para o desenvolvimento de um programa para resolução de um problema são necessárias as seguintes etapas:



LINGUAGEM DE PROGRAMAÇÃO

- Portanto, um programa é a codificação de um algoritmo em uma linguagem de programação (ASCENCIO, 1999);

Conceitos para algoritmo:

- Sequência de passos que visa atingir um objetivo bem definido (FORBELLONE, 1999);
- Descrição de uma sequência de passos que deve ser seguida para a realização de uma tarefa (ASCENCIO, 1999);
- Regras formais para a obtenção de um resultado ou da solução de um problema, englobando fórmulas de expressões aritméticas (MANZANO, 1997);

LINGUAGEM DE PROGRAMAÇÃO

Executamos no dia a dia vários algoritmos:

- **Algoritmo 1:** Fazer uma ligação telefônica

  Passo 1: Retirar o telefone do gancho;
  Passo 2: Esperar o sinal telefônico;
  Passo 3: Discar o número;
  Passo 4: Falar ao telefone:
  Passo 5: Encerrar a conversa;
  Passo 6: Encerrar a ligação (telefone no gancho)

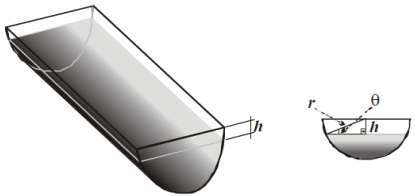
- **Algoritmo 2:** Somar três números reais

  Passo 1: Receber os três números;
  Passo 2: Somar os três números;
  Passo 3: Mostrar o resultado obtido

LINGUAGEM DE PROGRAMAÇÃO

- **Algoritmo 3:** Resolver, através do método da secante, o seguinte problema:

Um tanque de comprimento L, tem uma secção transversal no formato de um semicírculo com raio r conforme a figura abaixo.



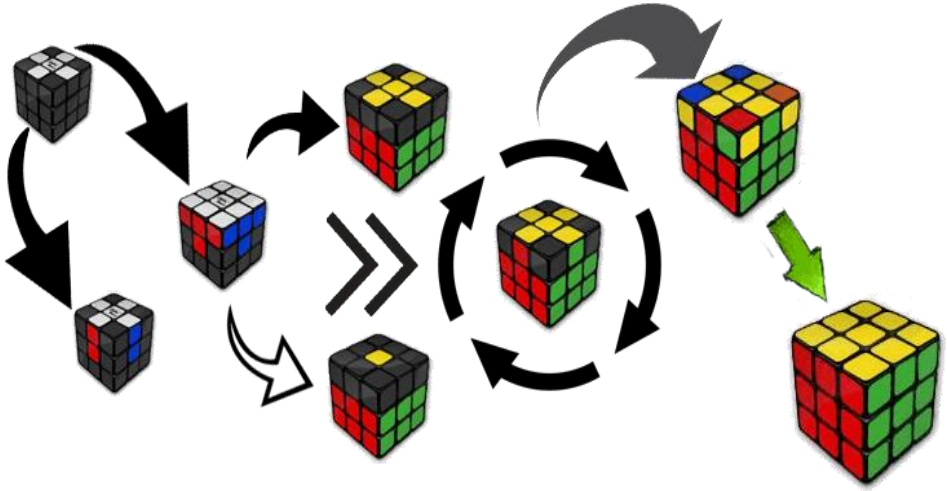
Quando cheio de água até uma distância h do topo o volume V da água é dado por:

$$V = L\left[0.5\,\pi r^2 - r^2 arcsen\left(\frac{h}{r}\right) - h\sqrt{r^2 - h^2}\right]$$

Supondo que $L = 10\,m$, $r = 1\,m$ e $V = 12.4\,m^3$, determine uma aproximação para o valor da profundidade da água no tanque.

LINGUAGEM DE PROGRAMAÇÃO

- Para a construção de um programa é necessário um algoritmo, e para a construção de um algoritmo utilizamos a **LÓGICA.**



https://youtu.be/YL2AzBYoPSs

LINGUAGEM DE PROGRAMAÇÃO

PROBLEMA 01: Você tem dois baldes: um com capacidade para comportar 5 litros, e outro que comporta 3 litros. Você não possui outros recipientes e os baldes não possuem marcações de volume.

Escreva um algoritmo para retirar exatamente sete litros de água de uma bica, com esses baldes.

MÉTODO PARA A CONSTRUÇÃO DE ALGORITMOS

Para a construção de qualquer tipo de algoritmo é necessário:

- Compreender completamente o problema a ser resolvido;
- Definir os dados de entrada, ou seja, quais dados serão fornecidos e quais objetos fazem parte do problema;
- Definir quais cálculos serão efetuados e quais as restrições para esses cálculos. O processamento é responsável pela transformação dos dados de entrada em dados de saída;
- Definir os dados de saída, ou seja, quais os tipos de dados esperados depois do processamento;
- Construir o algoritmo (escolher um dos tipos);
- Testar o algoritmo através de simulações.

ALGUMAS SUGESTÕES:

- **Não se aprende a construir algoritmos copiando algoritmos já prontos**;
- Uma **boa técnica** para construção de algoritmos é **construindo e testando os mesmos**;
- O algoritmo é "**apenas**" uma **sugestão de caminho** para a **solução do problema**.
- Um mesmo **problema pode ser resolvido** com o "**auxílio**" de **diferentes algoritmos.**

TIPOS DE ALGORITMOS

Os tipos mais utilizados de algoritmos são:

- ***DESCRIÇÃO NARRATIVA:*** escrever o algoritmo utilizando uma linguagem natural (por exemplo, a língua portuguesa).
  - ***Vantagem:*** não é necessário aprender nenhum conceito novo, pois, uma língua natural, neste ponto já é bem conhecida
  - ***Desvantagem:*** Neste caso, abrimos espaço para várias interpretações, o que posteriormente dificultará a transcrição desse algoritmo para o programa;

TIPOS DE ALGORITMOS

- ***FLUXOGRAMA (DIAGRAMA DE FLUXO):*** Consiste em analisar o problema e escrever o algoritmo utilizando símbolos gráficos predefinidos.

  - ***Vantagem:*** o entendimento de elementos gráficos é mais simples que o entendimento de textos;

  - ***Desvantagem:*** é necessário aprender a simbologia dos fluxogramas. O algoritmo resultante não apresenta muitos detalhes, dificultando a transcrição para um programa

FLUXO DE DADOS
Indica o sentido do fluxo de dados. Conecta os demais símbolos

TERMINAL
Indica o INÍCIO ou FIM de um processamento
Exemplo: Início do algoritmo

PROCESSAMENTO
Processamento em geral
Exemplo: Calculo de dois números

ENTRADA/SAÍDA (Genérica)
Operação de entrada e saída de dados
Exemplo: Leitura e Gravação de Arquivos

DESVIO (conector)
Permite o desvio para um ponto qualquer do programa

ENTRADA MANUAL
Indica entrada de dados via Teclado
Exemplo: Digite a nota da prova 1

EXIBIR/SAÍDA
Mostra informações ou resultados
Exemplo: Mostre o resultado do cálculo

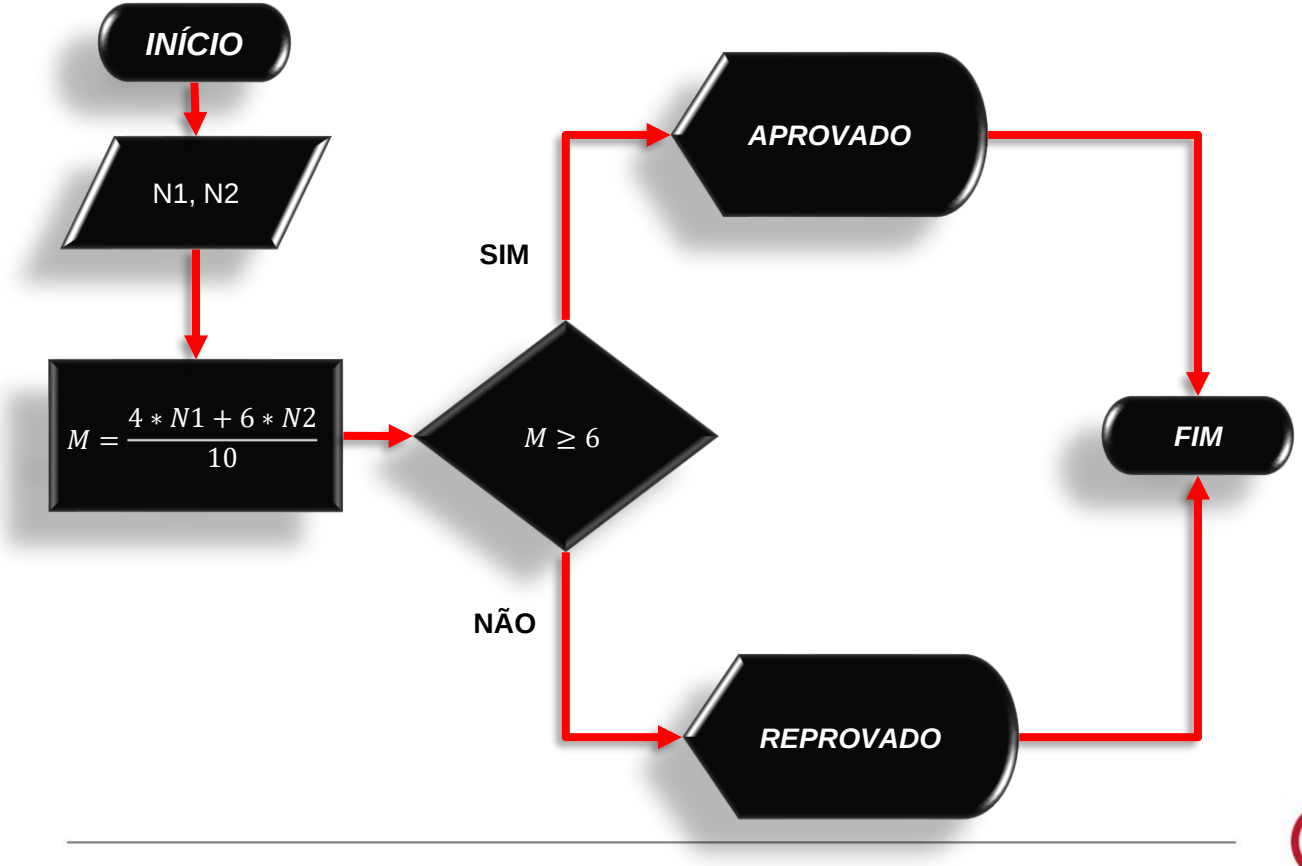
DECISÃO
Permite elaborar processos de decisão

CONECTOR DE PAGINA
Permite informar de qual pagina vem o fluxograma

Conjuntos de símbolos utilizados no fluxograma

GRUPO TIRADENTES
Inspirando o futuro

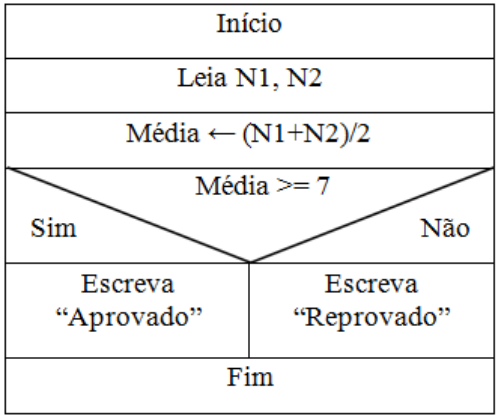
TIPOS DE ALGORITMOS



GRUPO TIRADENTES
Inspirando o futuro

TIPOS DE ALGORITMOS

- Para representação de algoritmos através de fluxogramas, existem também os chamados diagramas de Nassi-Shneirderman (Diagrama de Chapin). Por exemplo:



A ideia básica deste diagrama é representar as ações de um algoritmo dentro de um único retângulo, subdividindo-o em retângulos menores, que representam os diferentes blocos de sequência de ações do algoritmo.

TIPOS DE ALGORITMOS

- ***PSEUDOCÓDIGO OU PORTUGOL OU PORTUGUÊS ESTRUTURADO:*** Consiste em analisar o enunciado do problema e escrever por meio de regras predefinidas, os passos a serem seguidos para sua resolução.

  - *Vantagem:* a passagem do algoritmo para qualquer linguagem de programação é quase imediata;

  - Desvantagem: é necessário aprender as regras do pseudocódigo

- O PORTUGOL, é uma forma especial de linguagem bem mais restrita que a Língua Portuguesa e com significados bem definidos para todos os termos utilizados nas instruções (comandos).

TIPOS DE ALGORITMOS

- O Português Estruturado na verdade é uma simplificação extrema da língua portuguesa, limitada a pouquíssimas palavras e estruturas que têm significado pré-definido, pois deve-se seguir um padrão. Emprega uma linguagem intermediária entre a linguagem natural e uma linguagem de programação, para descrever os algoritmos.

TIPOS DE ALGORITMOS

EXEMPLO DO “PORTUGOL” ATRAVÉS DO VISUALG

```
1  algoritmo "SOMANDO VALORES"
2
3  var
4  N1: inteiro
5  N2: inteiro
6  M: inteiro
7
8  inicio
9
10 N1<-44
11 N2<-32
12 M<-N1 + N2
13
14 escreva("RESULTADO DA SOMA:  ", M)
15 fimalgoritmo
```

GRUPO TIRADENTES
Inspirando o futuro

EXEMPLOS DE ALGORITMOS

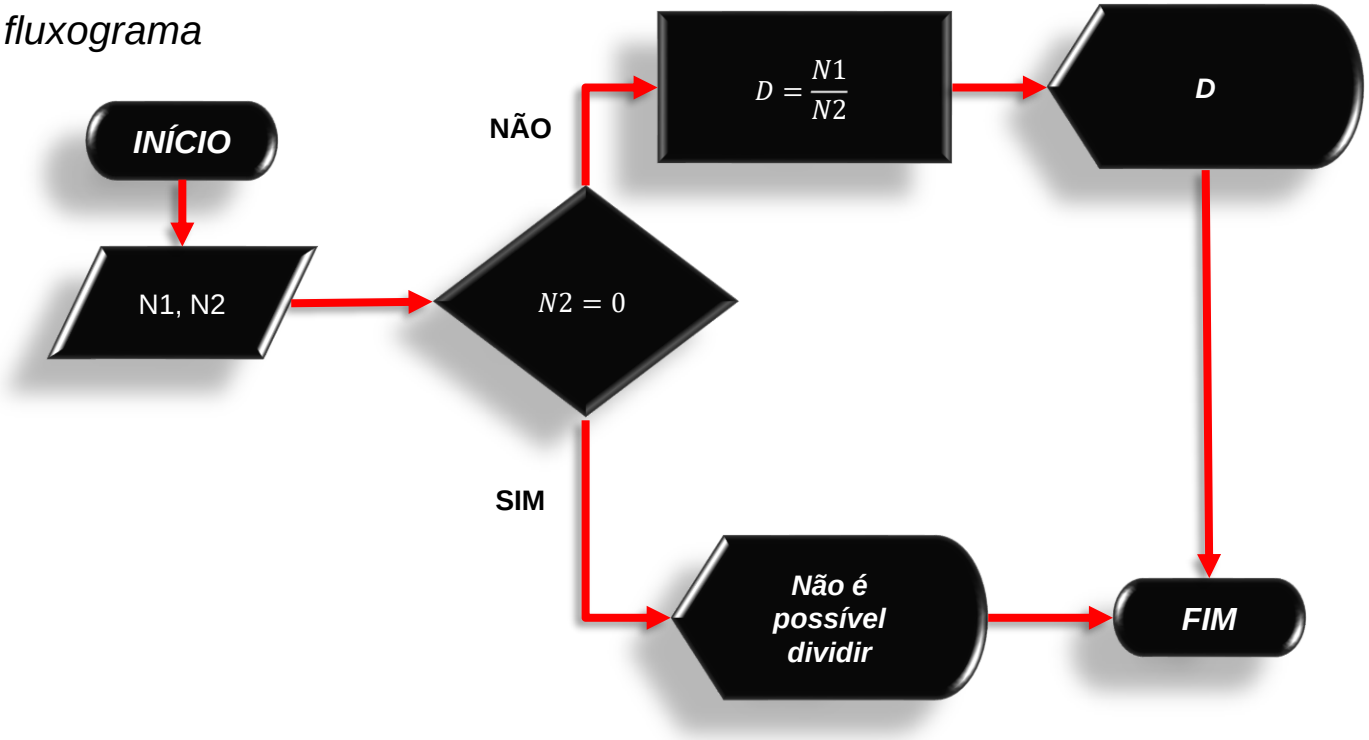
PROBLEMA 2: Mostrar o resultado da divisão do número real N1 pelo número real N2.

Algoritmo em descrição narrativa

- ***Passo 1:*** Receber os números reais N1 e N2;
- ***Passo 2:*** Se N2 for igual a zero, não poderá ser feita a divisão e assim temos o fim do algoritmo senão vá para o passo 3;
- ***Passo 3:*** Calcular N1/N2.

EXEMPLOS DE ALGORITMOS

Algoritmo em fluxograma



EXEMPLOS DE ALGORITMOS

Algoritmo em portugol (pseudocódigo)

```
1  algoritmo "DIVIDINDO VALORES REAIS"
2  var
3  N1: real
4  N2: real
5  D:  real
6
7  inicio
8  escreva("Entre com o valor de N1:  ")
9  leia (N1)
10 escreva("Entre com o valor de N2:  \\")
11 leia (N2)
12
13 se N2 = 0 entao
14      escreva("IMPOSSIVEL DIVIDIR  ")
15 senao
16    D <- N1/N2
17    escreva("   ")
18    escreva("RESULTADO DA DIVISAO:  ", D)
19 fimse
20 fimalgoritmo
```

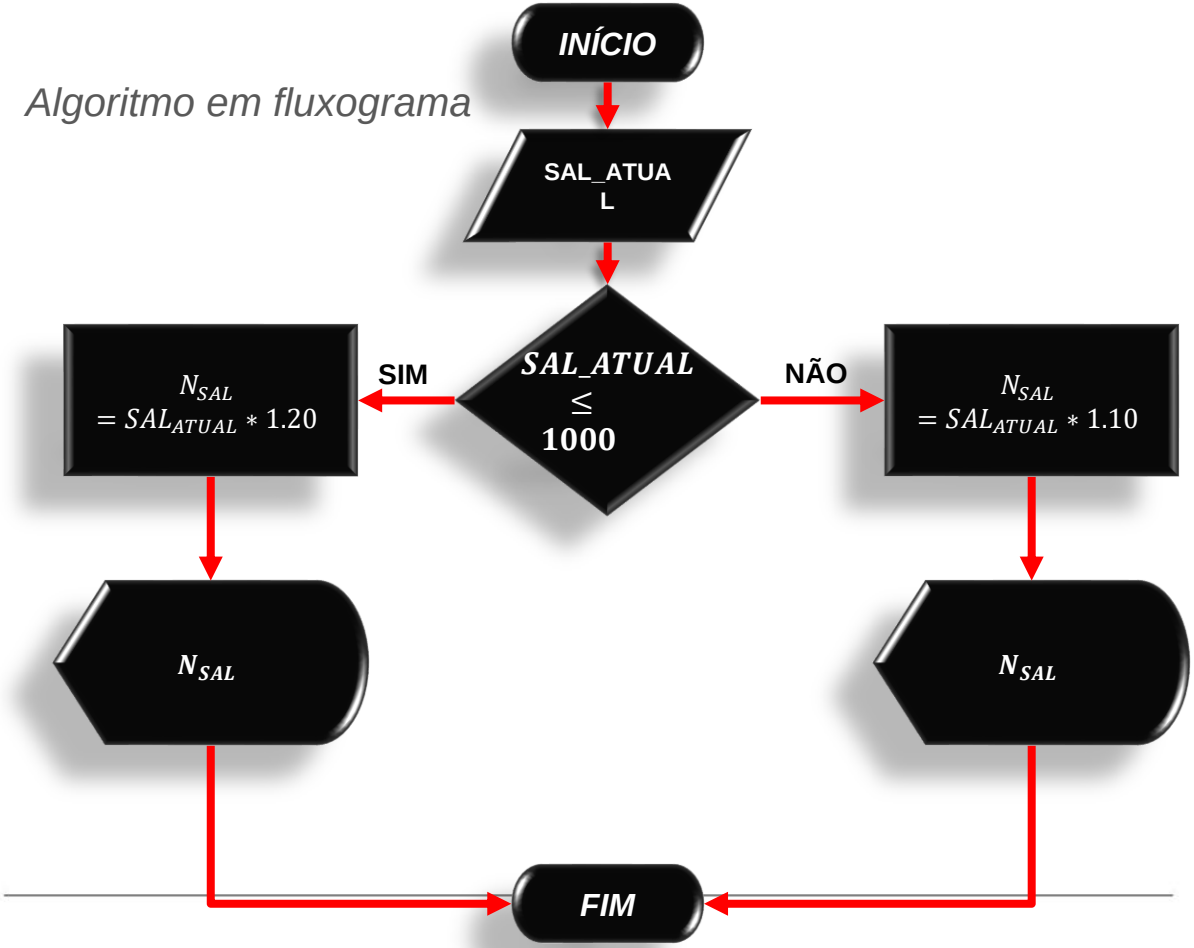
GRUPO TIRADENTES
Inspirando o futuro

EXEMPLOS DE ALGORITMOS

PROBLEMA 3: Faça os algoritmos em fluxograma e pseudocódigo para calcular o novo salário de cada funcionário de uma empresa. Sabe-se que os funcionários que recebem atualmente, salário de até R$ 1.000,00 terão aumento de 20% e os demais terão aumento de 10%.

EXEMPLOS DE ALGORITMOS

Algoritmo em fluxograma



GRUPO TIRADENTES
Inspirando o futuro

EXEMPLOS DE ALGORITMOS

Algoritmo em pseudocódigo

```
1  algoritmo "NOVO SALARIO"
2
3  var
4  sal_atual: real
5  novo_sal: real
6  inicio
7  escreva("VALOR DO SALARIO ATUAL:  ")
8  leia(sal_atual)
9
10 se sal_atual <= 1000 entao
11    novo_sal <- sal_atual * 1.20
12 senao
13    novo_sal <- sal_atual * 1.10
14 fimse
15
16 escreva("NOVO SALARIO:  ", novo_sal)
17 fimalgoritmo
```

GRUPO TIRADENTES
Inspirando o futuro

VARIÁVEL

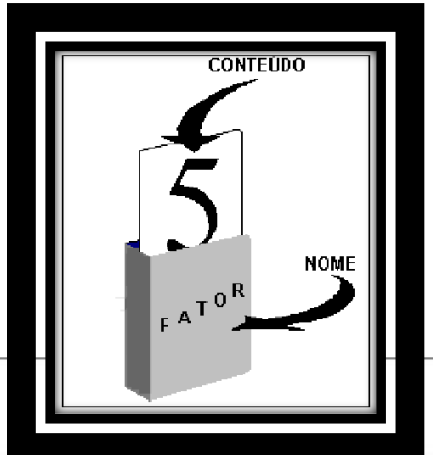
- Uma variável, é um espaço da memória do computador que "reservamos" para guardar informações (dados). Como o próprio nome sugere, as variáveis, podem conter valores diferentes a cada instante de tempo, ou seja, seu conteúdo pode variar de acordo com as instruções do algoritmo.

- As variáveis são referenciadas através de um nome (identificador) criado por você durante o desenvolvimento do algoritmo. Exemplos de nomes de variáveis: produto, idade, a, x, nota1, peso, preço, etc.

VARIÁVEL

- Uma variável armazena 'apenas' um conteúdo de cada vez. Uma variável pode ser vista como uma caixa com um rótulo (nome) colado nela, que em um dado momento guarda um determinado objeto. O conteúdo desta caixa não é algo fixo, permanente.

- No exemplo abaixo, a caixa (variável) rotulada como FATOR, contém o valor 5. Em outro momento essa caixa poderá conter qualquer outro valor numérico. Entretanto, a cada instante, ela conterá um, e somente um, valor.



GRUPO TIRADENTES
Inspirando o futuro

TIPOS DE DADOS

Os tipos de dados utilizados em algoritmos são:

- ***NUMÉRICOS:*** Dividem-se em dois grupos: inteiros e reais.

- ***LÓGICOS:*** são também chamados de dados Booleanos (Álgebra d Boole), e podem assumir os valores verdadeiro ou falso.

- ***LITERAIS OU CARACTERES:*** São dados formados por um único caractere ou por uma cadeia de caracteres. Estes caracteres podem ser letras maiúsculas, minúsculas, números (não podem ser utilizados para cálculos) e os caracteres especiais (&,#,@,?,...).

OBS: Um caractere é representado entre apóstrofos e um conjunto de caracteres é representado entre aspas.

TIPOS DE DADOS

EXEMPLO: Informações relativas a um funcionário de uma empresa:

Nome	José da Silva
Idade	34
Sexo	M
Salário	R$7.500,00

IDENTIFICADORES

- São os nomes das variáveis, dos programas, das constantes, das rotinas, das unidades, etc. As regras básicas para a formação dos identificadores são:
  - os caracteres que você pode utilizar na formação dos identificadores são: A. os números; B. o caractere sublinhado; C. as letras maiúsculas e minúsculas;
  - o primeiro caractere deve ser sempre uma letra ou o caractere sublinhado;
  - não são permitidos espaços em branco e caracteres especiais (@, $, +, –, %, !);
  - não podemos usar palavras reservadas, ou seja, palavras que pertencem a uma linguagem de programação.

IDENTIFICADORES

A
nota
NOTA
X4
A32
nota_1
IDADE

Exemplos de identificadores válidos

5B	Começa com número
E 5	Contém espaço em branco
x-y	Contém caractere especial
prova 2n	Contém espaço em branco
case	Palavra reservada

Exemplos de identificadores NÃO válidos

ATRIBUIÇÃO

- A atribuição é uma notação utilizada para atribuir um valor a uma variável, ou seja, para armazenar um determinado conteúdo em uma variável.

- A operação de atribuição, normalmente, é representada por uma seta apontando para a esquerda, mas existem outros símbolos para representar a atribuição, depende da forma de representação do algoritmo.

Atribuições Possíveis	Exemplos
variável ← constante	idade ← 12 (lê-se: idade *recebe* 12)
variável ← variável	preço ← valor
variável ← expressão	A ← B + C

ATRIBUIÇÃO

ATENÇÃO: Nas atribuições é a parte esquerda que irá receber algo. Assim, no lado esquerdo não pode haver nada além da variável. Assim, por exemplo, a atribuição:

$$nota_1 + nota_2 \longleftarrow valor$$

não é válida !!!

EXPRESSÕES

- Conjunto de variáveis e/ou valores separados por caracteres especiais, que indicam as operações que devem ser executadas.

- As operações básicas utilizadas em uma expressão podem ser classificadas em:
  - Aritméticas
  - Relacionais
  - Lógicas

EXPRESSÕES

OPERADORES ARITMÉTICOS NO PYTHON

Operador	Descrição	Exemplo
>	Maior que	2 > 3
<	Menor que	2 < 4
>=	Maior igual que	2 >= 2
<=	Menor Igual que	4 <= 4
!=	Diferente de	True != False
==	Igual a	True == 1

EXPRESSÕES

Hierarquia das Operações Aritméticas

- Dentro de uma mesma operação aritmética podem aparecer várias operações. Estas operações serão executadas de acordo com a seguinte ordem:

(1) O que estiver entre parênteses;

(2) Exponenciação;

(3) Multiplicação, divisão real, divisão inteira, ou resto da divisão inteira;

(4) Adição ou subtração;

EXPRESSÕES

IMPORTANTE: Quando existem duas ou mais operações da mesma hierarquia, estas são executadas na ordem em que aparecem na expressão, da esquerda para direita.

Exemplo: Considere a expressão:

$$y = \frac{2 + a}{b - 3} - 2x + x^3$$

No Python:

```
y=[(2+a)/(b-3)]-2*x + x**3
```

EXPRESSÕES

► OPERADORES RELACIONAIS:

Operador	Descrição	Exemplo
>	Maior que	2 > 3
<	Menor que	2 < 4
>=	Maior igual que	2 >= 2
<=	Menor Igual que	4 <= 4
!=	Diferente de	True != False
==	Igual a	True == 1

EXPRESSÕES

- ***OPERADORES LÓGICOS:*** Vamos considerar apenas os operadores lógicos:

E, OU, NÃO, XOU

- Os operadores lógicos Ou e E são aplicados sempre a dois operandos.
- O operador lógico NÃO é aplicado a um único operando
- Os operandos podem ser expressões desde que tenham como resultado um valor lógico (verdadeiro ou falso)

EXPRESSÕES

OPERADORES LÓGICOS:

Operador	Descrição	Exemplo
not	Negação Lógica	not True
and	E lógico	True and False
or	OU lógico	False or True

Atividades Práticas

PROBLEMA 1: Escreva um algoritmo através do fluxograma e do pseudocódigo (portugol) que recebe 3 valores reais, e calcula a média aritmética entre estes valores.

PROBLEMA 2: Escreva um algoritmo através do fluxograma e do pseudocódigo (portugol) que recebe três notas, e seus respectivos pesos e calcula e apresenta a média ponderada entre estas notas.

PROBLEMA 3: Escreva um algoritmo através do fluxograma e do pseudocódigo (portugol) que recebe o salário do funcionário, calcula e mostra o novo salário sabendo que este teve um aumento de 25%.

INTRODUÇÃO AO PYTHON

GRUPO TIRADENTES
Inspirando o futuro

SOBRE O PYTHON:

- Linguagem de programação de altíssimo nível (VHLL – Very High Level Language), criada pelo holandês Guido Van Rossun, no início de 1989 sob o ideal de “Programação de Computadores para todos”. A primeira versão do Python foi liberada ao público em 1991.

- É uma linguagem de programação interpretada, de código fonte aberto e disponível para vários sistemas operacionais;

- Origem do nome: grupo de humoristas ;

SOBRE O PYTHON:

- Sintaxe simples e fácil de ser assimilada;
- Fácil de usar, aprender, ler;
- Orientada à objetos, estruturada e funcional;
- Tipagem forte e dinâmica;
- Ambiente interativo

SOBRE O PYTHON:

- Extremamente portável (Multiplataforma);
- Compilação implícita e automática;
- Gerenciamento automático memória (Garbage Collector);
- Poderosas estruturas de dados nativas;
- Pacotes, módulos, classes, funções;
- Identação para estrutura de bloco

ALGUNS USUÁRIOS DO PYTHON:

- Google (vários projetos)
- NASA (vários projetos)
- RedHat (ferramentas instalação Linux)
- Muitas Universidades, como MIT, e Stanford
- Globo.com

SITE DO PYTHON:

https://www.python.org

TIPOS DE VARIÁVEIS:

Variáveis são formas de se armazenar dados para uso posterior, elas podem ser classificadas em 3 tipos básicos que são mostradas logo abaixo. Quando analisarmos as listas veremos que existem outras variáveis mais complexas.

- *int* - Um número inteiro
- *float* - Um ponto flutuante
- *string* - Uma sequência de caracteres

STRINGS:

- Tipo de objeto formado por uma sequência imutável de caracteres que nos permite trabalhar com textos.

Exemplo:

```
1 a = "boa noite"
2 print(a)
3
4
```

Strings:

- Pelo fato de uma string ser uma sequência imutável, isso nos da a possibilidade de manipularmos essa sequência

Exemplo:

```
a = "matemática"
w = a[2]+a[3]+a[0]
print(w)
```

Strings:

- Existe também um recurso conhecido como operador %, que serve para formatar as strings, basicamente são três os tipos de formatação que temos:

  - %s - serve para substituir string;
  - %d - serve para substituir números inteiros em uma frase destinada a um print;
  - %f - serve para substituir floats (números em aritmética de ponto flutuante).

Strings:

EXEMPLO:

```
1 compra = "maça"
2 tipo = "verde"
3 quilos = 1.5
4
5 print("Maria comprou %f quilos de %s %s" %(quilos,compra,tipo))
```

Formatação de Números:

EXEMPLO:

```
1 area = 345.787965854674
2 print("area = %.4f" %area)
3
```

OPERADORES ARITMÉTICOS:

Operação aritmética	Operador
adição	+
subtração	-
multiplicação	*
divisão	/
exponenciação	**
parte inteira (da divisão)	//
módulo (resto da divisão)	%

OPERADORES RELACIONAIS:

Operação relacional	Operador
Igualdade	==
Diferença	!= ou <>
Maior	>
Maior ou igual que	>=
Menor	<
Menor ou igual que	<=

ENTRADA DE DADOS

COMANDO:
input()

```
>>> idade = input('Digite sua idade: ')
Digite sua idade: 27
>>> altura = input('Qual a sua altura: ')
Qual a sua altura: 1.75
```

RESUMO DE COMANDOS BÁSICOS:

Conceitos Básicos	Python
Nome das Variáveis	Regra básica: somente letras (do alfabeto) sem acentuação e sem espaços. Ex: nota1, NOTA2, nOTa3, Nota4.
Entrada de Dados (Comando)	**input** sendo armazenado / atribuído a uma variável. Ex: nota1 = input("Qual a nota do aluno? ")
Saída de dados (Comando)	**print** contendo textos e variáveis. Ex: print("A nota do aluno foi", nota1)

INTRODUÇÃO AO PYTHON

PROF. DR. CARLOS GUSTAVO P. MORAES

2022 - ANO DA realização

GRUPO TIRADENTES
Inspirando o futuro

A FUNÇÃO print()

A função *print( )*, basicamente, “imprime” na tela, ou seja, exibe, aquilo que está dentro dos parênteses.

```
1 print('Oi Gustavo')
```

```
Oi Gustavo
```

```
1 print("Oi Gustavo")
```

```
Oi Gustavo
```

COMENTÁRIOS

Comentários são auto explicativos.

São anotações no código que não são rodadas junto com o próprio, e servem para ilustrar, explicar ou incluir informações adicionais nele.

No Python existem duas formas de comentários. O primeiro, o comentário de uma linha, deve ser antecedido por “#”, e transforma em comentário tudo que está a partir dele, até o final daquela linha.

E ele não precisa estar no começo da linha, ele pode começar logo após alguma linha de código. Veja exemplos:

```
1  # Isto é um comentário e não será executado pelo Python
2  print("Oi Alunos")
3  print("Explicando comentários") # isto também é um comentário e não será executado
```

```
Oi Alunos
Explicando comentários
```

TIPOS DE DADOS

O Python permite trabalhar com diferentes tipos de dados. Um destes tipos é o tipo **String.** Vamos passar por cada tipo, explicando cada um deles:

- Strings - são dados definidos entre aspas duplas (“) ou simples (‘), representando texto, que normalmente é exibido por um programa ou exportado por ele.

  - Ex: “Oi Python”

TIPOS DE DADOS

- Números Inteiros (Integer) - são números sem casas decimais, números inteiros. São definidos simplesmente como um número inteiro comum.
  - Ex: 5, 31, 150

- Números decimais (float) - números com casas decimais, são definidos com o número e as casas decimais após o ponto.
  - Ex: 5.67, 99.99

TIPOS DE DADOS

- Boolean - utilizado para representar os conceitos de Verdadeiro ou Falso.

Booleans só podem assumir estes dois valores, representados por “True” e “False” (mas sem as aspas)

- Ex: True, False

Temos também outros tipos de dados, como Listas (lists), Dicionários (dictionaries) e Tuplas (tuples), entre outros.

SINAIS MATEMÁTICOS BÁSICOS

Vamos agora falar um pouco de sinais e operações em Python. Para fazer a representação das operações, vamos utilizar números inteiros (integer), mas todas elas também funcionam com números decimais (float). Para as operações matemáticas básicas, o Python possui os seguintes sinais:

Sinal	Operação	Exemplo	Definição
+	Soma	>>> 4+5 9	Soma dois números
-	Subtração	>>> 6-3 3	Subtrai dois números
*	Multiplicação	>>> 7*8 56	Multiplica dois números
/	Divisão	>>> 12/4 3.0	Divide dois números
%	Resto da Divisão	>>> 63 % 10 3	Retorna o resto da divisão inteira entre dois números
**	Potenciação	>>> 5**3 125	Eleva o número antes do sinal à potência do número após

GRUPO TIRADENTES
Inspirando o futuro

0s

```python
print(4 + 5)
print(6 - 3)
print(7 * 8)
print(12 / 5)
print(63 % 10)
print(5**3)
```

```
9
3
56
2.4
3
125
```

GRUPO TIRADENTES
Inspirando o futuro

SINAIS DE COMPARAÇÃO

Existem também os sinais de comparação. Estes sinais retornarão True ou False, indicando se uma determinada comparação é Verdadeira ou Falsa. True e False são valores especiais em Python, um tipo de dado chamado Boolean, onde os únicos valores possíveis são estes dois. Vamos aos sinais de comparação:

Sinal	Operação	Exemplo	Definição

==	Igual a	>>> 4 == 5
		False
		>>> 4 == 4
		True
			Indica se dois valores são iguais

!=	Diferente de	>>> 6 != 3
		True
		>>> 5 != 5
		False
			Indica se dois valores não são iguais

>	Maior que	>>> 9 > 8
		True
		>>> 3 > 5
		False
			Indica se o número anterior ao sinal é maior que o posterior

>=	Maior ou igual a	>>> 10 >= 10
		True
		>>> 13 >= 15
		False
			Indica se o número anterior ao sinal é maior ou igual ao posterior

<	Menor	>>> 63 < 65
		True
		>>> 150 < 140
		False
			Indica se o número anterior ao sinal é menor que o posterior

<=	Menor ou igual a	>>> 5 <= 5
		True
		>>> 12 <= 8
		False
			Indica se o número anterior ao sinal é menor ou igual ao posterior

0s

```python
print(4 == 5)
print(4 == 4)
print(6 != 3)
print(6 != 6)
print(9 > 8)
print(3 > 5)
print(10 >= 10)
print(13 >= 15)
print(63 < 65)
print(150 < 140)
print(5 <= 5)
print(12 <= 8)
```

```
False
True
True
False
True
False
True
False
True
False
True
False
```

VARIÁVEIS

As variáveis são elementos que nos permitem guardar valores para reutilizar posteriormente. Imagine que você fez algum cálculo complexo com dados recuperados de algum outro lugar. **Agora** imagine que você deseja usá-lo posteriormente em seu programa. Não faz sentido fazer recuperar todas as informações e fazer o cálculo todo novamente. Para isso, usamos as variáveis.

DEFININDO VARIÁVEIS

Para definir uma variável no Python, basta digitar um nome para a própria, o sinal de igual e o valor que se deseja que ela armazene.
Uma variável em Python pode armazenar praticamente qualquer coisa. Números, strings, listas, dicionários.
Os nomes de variáveis em Python precisam começar com uma letra ou com um underscore "_". Números podem ser usados se não estiverem no começo. Variáveis em Python também não possuem tipos definidos. Desta forma, uma mesma variável que armazena inicialmente uma string, pode depois armazenar um número inteiro, ou uma lista. Vamos então a um exemplo de definição de variáveis:

0s

```
a = 5
b = 7
print(a+b)
a = 'Carlos Gustavo'
print(a)
```

```
12
Carlos Gustavo
```

GRUPO TIRADENTES
Inspirando o futuro

Você também pode designar a uma variável o valor de outra variável. Prosseguindo com o exemplo:

```
b = 'Carlos Gustavo'
print(b)
```

```
Carlos Gustavo
```

Também é possível definir mais de uma variável de uma vez. Caso o valor seja igual para todas as variáveis, utilize o sinal de igual entre os nomes das variáveis. Caso seja diferente, use vírgulas entre os nomes das variáveis e entre os valores de cada uma delas.
Veja os exemplos abaixo:

```python
# Definição de múltiplas variáveis com valores iguais
x = y = z = 10
print(x)
print(y)
print(z)
# Definição de múltiplas variáveis com valores diferentes
x, y, z = 10, 20, 30
print(x, y, z)
```

```
10
10
10
10 20 30
```

A FUNÇÃO input( )

Esta função é utilizada para captar informações enviadas pelo usuário. Ela pode receber um parâmetro, que será uma mensagem exibida ao usuário antes que ele insira o que deseja. A entrada inserida pelo usuário pode ser salva e utilizada posteriormente.

```
1 nome = input("Olá, qual é seu nome? ")
2 print("Olá", nome)
```

```
Olá, qual é seu nome? Carlos Gustavo
Olá Carlos Gustavo
```

```
[16]  1 nome = input("Olá, qual é seu nome?\n")
      2 print("Olá, %s " % nome)
```

```
Olá, qual é seu nome?
Carlos Gustavo
Olá, Carlos Gustavo
```

STRINGS

Definindo Strings

Uma string é definida como tudo que se encontra entre aspas simples (‘) ou duplas (“). Qual você usa é uma questão de gosto e opinião pessoal. Desta forma vamos aos primeiros exemplos:

```
string1 = "Oi Gustavo"
string2 = "Tchau Gustavo"
print(string1)
print(string2)
```

```
Oi Gustavo
Tchau Gustavo
```

Existe uma terceira forma, que é a definição de strings entre três aspas triplas. Neste caso, caracteres escapados assumem seu significado correto, e quebras de linha, seja no terminal ou na linha de código também são interpretadas como tal no output. Veja o exemplo abaixo, onde cada quebra de linha foi feita dando ENTER após cada sentença. A string só se encerra ao fechá-la com o mesmo conjunto de aspas simples ou duplas utilizado para iniciá-la:

```python
string_grande = '''Olá. Esta é uma string grande no Python.
Aqui você pode usar " ou '
Caracteres são escapados como se espera.
É a terceira forma de definir uma string, apesar de não ser tão usual....
\t testando o TAB para finalizar'''

print(string_grande)
```

```
Olá. Esta é uma string grande no Python.
Aqui você pode usar " ou '
Caracteres são escapados como se espera.
É a terceira forma de definir uma string, apesar de não ser tão usual....
         testando o TAB para finalizar
```

ESCAPANDO STRINGS

```python
string3 = "A cantora Sinnead O'Connor"
string4 = 'LUIZ GOMES disse "ESTUDEM!"'
print(string3)
print(string4)
```

```
A cantora Sinnead O'Connor
LUIZ GOMES disse "ESTUDEM!"
```

0s

```python
string4 = 'LUIZ GOMES disse \'ESTUDEM!\''
print(string4)
```

```
LUIZ GOMES disse 'ESTUDEM!'
```

Sequência Escapada	Significado
\\	Contrabarra
\'	Aspas simples
\"	Aspas duplas
\a	Caracter de controle ASCII Bell (BEL)
\b	Caracter ASCII Backspace (BS)
\f	Caracter ASCII Formfeed (FF)
\n	Caracter ASCII Linefeed (LF) - este cria uma nova linha no output
\r	Caracter ASCII Carriage Return (CR)
\t	Caracter ASCII Tab Horizontal (TAB)
\v	Caracter ASCII Tab Vertical (VT)
\ooo	Caracter ASCII com valor octal "ooo"
\xhh	Caracter ASCII com valor hex de "hh"

GRUPO TIRADENTES
Inspirando o futuro

```python
a = 30.46257
print("Formato decimais: %f" % a)
print("Formato decimais: %.2f" % a)
print("Formato decimais: %.3f" % a)
```

```
Formato decimais: 30.462570
Formato decimais: 30.46
Formato decimais: 30.463
```

STRINGS COMO LISTAS

Em Python, toda string é tratada como uma lista.

```
string9 = "Olá, meu nome é Luiz Gomes"
print(string9[0])
print(string9[5])
print(string9[21])
```

```
O
m
G
```

FUNÇÕES ÚTEIS PARA STRINGS

capitalize() – transforma a primeira letra em maiúscula.
lower() – transforma todas as letras em minúsculas.
upper() – transforma todas as letras em maiúsculas.

```
string10 = "olá, meu NOME é LUIZ GOMES DA cunha neto"
print(string10.capitalize())
print(string10.lower())
print(string10.upper())
```

```
Olá, meu nome é luiz gomes da cunha neto
olá, meu nome é luiz gomes da cunha neto
OLÁ, MEU NOME É LUIZ GOMES DA CUNHA NETO
```

FUNÇÕES ÚTEIS PARA STRINGS

center(), ljust() e rjust()

```python
string10 = "olá, meu NOME é LUIZ GOMES DA cunha neto"
print(string10.center(50,"#"))
print(string10.ljust(50,"#"))
print(string10.rjust(50,"#"))
```

```
#####olá, meu NOME é LUIZ GOMES DA cunha neto#####
olá, meu NOME é LUIZ GOMES DA cunha neto##########
##########olá, meu NOME é LUIZ GOMES DA cunha neto
```

```python
string10 = "olá, meu NOME é LUIZ GOMES DA cunha neto"
print(string10.center(50," "))
print(string10.ljust(50,"#"))
print(string10.rjust(50,"#"))
```

```
     olá, meu NOME é LUIZ GOMES DA cunha neto
olá, meu NOME é LUIZ GOMES DA cunha neto##########
##########olá, meu NOME é LUIZ GOMES DA cunha neto
```

LISTAS

PROF. LUIZ GOMES DA CUNHA NETO

2022 - ANO DA realização

GRUPO TIRADENTES
Inspirando o futuro

LISTAS

As listas são estruturas de dados sequenciais do Python que servem para armazenar outros diferentes tipos de dados. As listas podem armazenar qualquer tipo de dado, como números inteiros, strings, números decimais e até outras listas.

CRIANDO, MODIFICANDO E ACESSANDO ITENS EM UMA LISTA

Para criar uma lista, basta definir seus itens entre colchetes “[ ]” e separados por vírgula. Também é possível criar uma lista vazia, definindo apenas os colchetes, sem nenhum item dentro deles. Vejamos alguns exemplos:

0s

```python
alunos = ["Antônio", "Clara", "Luiz"]
notas = [10.0, 9.2, 9.9]
print(alunos)
print(notas)
lista_vazia = []
print(lista_vazia)
print(alunos[0])
print(notas[2])
print(alunos[-1])
print(notas[-2])
```

```
['Antônio', 'Clara', 'Luiz']
[10.0, 9.2, 9.9]
[]
Antônio
9.9
Luiz
9.2
```

GRUPO TIRADENTES
Inspirando o futuro

ADICIONANDO NOVOS ITENS EM UMA LISTA

0s

```python
alunos = ["Antônio", "Clara", "Luiz"]
print(alunos)
alunos.append('Alfredo')
print(alunos)
alunos.insert(1, "Daniela")
print(alunos)
novos_alunos = ['Carlos', 'Maria', 'Ana']
alunos.extend(novos_alunos)
print(alunos)
```

```
['Antônio', 'Clara', 'Luiz']
['Antônio', 'Clara', 'Luiz', 'Alfredo']
['Antônio', 'Daniela', 'Clara', 'Luiz', 'Alfredo']
['Antônio', 'Daniela', 'Clara', 'Luiz', 'Alfredo', 'Carlos', 'Maria', 'Ana']
```

```python
alunos1 = ['José', 'Daniel', 'João']
alunos2 = ['Carlos', 'Augusto', 'Denis']
print(alunos1 + alunos2)
```

```
['José', 'Daniel', 'João', 'Carlos', 'Augusto', 'Denis']
```

```python
alunos = ["Antônio", "Clara", "Luiz"]
print(alunos)
alunos.remove('Clara')
print(alunos)
```

```
['Antônio', 'Clara', 'Luiz']
['Antônio', 'Luiz']
```

As listas também têm uma funcionalidade interessante, que é a de extrair partes da lista, o que em inglês é chamado de “slicing”. Para fazer isso, usamos uma notação semelhante à de acessar itens dentro das listas, mas informamos o intervalo que desejamos acessar. Vejamos o exemplo para que fique mais claro:

```python
alunos = ["Antônio", "Clara", "Luiz", "José", "Maria"]
print(alunos)
print(alunos[0:3])
print(alunos[2:4])
print(alunos[2:5])
```

```
['Antônio', 'Clara', 'Luiz', 'José', 'Maria']
['Antônio', 'Clara', 'Luiz']
['Luiz', 'José']
['Luiz', 'José', 'Maria']
```

Com esta notação, o Python pega os itens no intervalo definido dentro dos colchetes, iniciando o intervalo no primeiro número e terminando no segundo, mas sem incluir o segundo.

FUNÇÕES ÚTEIS PARA TRABALHAR COM LISTAS

len()

A função “ *len()*”, assim como ocorre quando utilizada com strings, retorna a quantidade de itens em uma lista:

```
alunos = ["Antônio", "Clara", "Luiz", "José", "Maria"]
print(len(alunos))
```

```
5
```

max() e min()

As funções “ *max()*” e “ *min()*”, como os nomes já devem indicar, retornam o valor máximo e o mínimo em uma lista:

```
notas = [1.31, 2.71, 3.5, 40 , 70, 3110.1]

print(notas)
print(min(notas))
print(max(notas))
```

```
[1.31, 2.71, 3.5, 40, 70, 3110.1]
1.31
3110.1
```

copy()

A função “ *copy()*” é a forma correta de criar uma cópia de uma lista. Definir uma nova lista igual a uma lista prévia (utilizando o sinal de “=”), apesar de parecer intuitivo, não funciona, pois isto cria apenas uma referência à lista. Vejamos primeiramente a forma errada:

```
alunos = ["José", "João", "Luiz", "Carlos", "Afonso"]
alunos_backup = alunos
print(alunos_backup)
alunos.clear()
print(alunos_backup)
```

```
['José', 'João', 'Luiz', 'Carlos', 'Afonso']
[]
```

Repare como a lista “alunos_backup” possuía os itens iguais à lista aluno, mas ficou vazia após usarmos a função “*c lear()*” para remover todos os itens da lista alunos. De nada adiantou fazer o backup desta forma. Vejamos agora a forma correta de criar uma cópia, usando a função “copy()” (vamos aproveitar e recriar a lista “alunos”, que ficou vazia após o último exemplo):

```
1  alunos = ['José', 'Daniela', 'Carla', 'Carlos', 'Augus
2  alunos_backup = alunos.copy()
3  print(alunos_backup)
4  alunos.clear()
5  print(alunos_backup)
```

```
['José', 'Daniela', 'Carla', 'Carlos', 'Augusto', 'Denis']
['José', 'Daniela', 'Carla', 'Carlos', 'Augusto', 'Denis']
```

DICIONÁRIOS

Os dicionários do Python, ou dictionaries (também referenciados como dicts), são mais uma estrutura para armazenar dados. Entretanto, o que difere significativamente os dicionários das listas é que no dicionário, o índice é um nome que você define, ao invés dos índices numerados que as listas possuem.

CRIANDO UM DICIONÁRIO, ACESSANDO E MODIFICANDO SUAS INFORMAÇÕES

Para criar um dicionário, usamos as chaves ({}). Os elementos seguem o formato “chave” : “valor”.

```
aluno = {"nome": "LUIZ GOMES", "idade": 20, "nota": 9.25}
print(aluno)
```

```
{'nome': 'LUIZ GOMES', 'idade': 20, 'nota': 9.25}
```

Para acessar os dados de um dicionário, usamos o mesmo formato das listas. Porém, como o índice é nomeado, usaremos o nome para resgatar o valor, ao invés do índice numérico:

```
aluno = {"nome": "LUIZ GOMES", "idade": 20, "nota": 9.25}
print(aluno["nome"])
print(aluno["idade"])
```

```
LUIZ GOMES
20
```

Se tentarmos acessar uma chave que não existe no dicionário, o Python retorna um erro:

```python
aluno = {"nome": "LUIZ GOMES", "idade": 20, "nota": 9.25}
print(aluno["nome"])
print(aluno["idade"])
print(aluno["Altura"])
```

```
LUIZ GOMES
20
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-3-bf3a48213d64> in <cell line: 4>()
      2 print(aluno["nome"])
      3 print(aluno["idade"])
----> 4 print(aluno["Altura"])

KeyError: 'Altura'
```

SEARCH STACK OVERFLOW

Para modificar um valor, novamente, fazemos como nas listas. Basta indicar a chave que se deseja alterar e definir o novo valor. A inclusão de um novo item no dicionário é feita da mesma forma, utilizando uma chave que ainda não foi utilizada no mesmo:

```
aluno = {"nome": "LUIZ GOMES", "idade": 20, "nota": 9.25}
aluno["nota"] = 8.2
aluno["peso"] = 74
print(aluno)
```

```
{'nome': 'LUIZ GOMES', 'idade': 20, 'nota': 8.2, 'peso': 74}
```

Repare como a chave “peso” e seu valor foram adicionados, além da alteração no valor da chave “nota”.

Para remover uma determinada chave e seu valor, usamos o “del”. Para apagar todas as chaves e valores, usamos o método “clear”, chamado através do dicionário:

```python
aluno = {"nome": "LUIZ GOMES", "idade": 20, "nota": 9.25}
aluno["nota"] = 8.2
aluno["peso"] = 74
print(aluno)
del aluno["idade"]
print(aluno)
aluno.clear()
print(aluno)
```

```
{'nome': 'LUIZ GOMES', 'idade': 20, 'nota': 8.2, 'peso': 74}
{'nome': 'LUIZ GOMES', 'nota': 8.2, 'peso': 74}
{}
```

FUNÇÕES ÚTEIS PARA DICIONÁRIOS

Algumas das funções que utilizamos com as listas também funcionam com os dicionários. É o caso da função “ **len()**”, por exemplo, que retorna a quantidade de pares chave : valor do dicionário.

```python
aluno = {"nome": "LUIZ GOMES", "idade": 20, "nota": 9.25}
print(len(aluno))
print(aluno)
del aluno["idade"]
print(len(aluno))
print(aluno)
```

```
3
{'nome': 'LUIZ GOMES', 'idade': 20, 'nota': 9.25}
2
{'nome': 'LUIZ GOMES', 'nota': 9.25}
```

Outra funcionalidade que também funciona com dicionários é a verificação da existência de uma chave através do “*in*”:

```
aluno = {"nome": "LUIZ GOMES", "idade": 20, "nota": 9.25}
print("idade" in aluno)
print("peso" in aluno)
```

```
True
False
```

get()

A função “ **get()**” é muito útil para evitar erros de chave inexistente. Ela recebe dois parâmetros, que são a chave a ser buscada e um valor de retorno. Ela retorna o valor desta chave se ela existir no dicionário, e se não existir, retorna aquilo que definimos como valor de retorno, ou None se nenhum valor de retorno for definido:

```python
aluno = {"nome": "LUIZ GOMES", "idade": 39, "nota": 9.2}
print(aluno.get("nome"))
print(aluno.get("peso"))
print(aluno.get("peso", "Não existe"))
```

```
LUIZ GOMES
None
Não existe
```

items(), keys() e values()

A função “**items()**” retorna uma lista de tuplas com os pares “chave” : valor.
A função “**keys()**” retorna uma lista apenas com as chaves do dicionário.
A função “**values()**” retorna uma lista apenas com os valores dos itens do dicionário:

```
aluno = {"nome": "LUIZ GOMES", "idade": 39, "nota": 9.2}
print(aluno.items())
print(aluno.keys())
print(aluno.values())
```

```
dict_items([('nome', 'LUIZ GOMES'), ('idade', 39), ('nota', 9.2)])
dict_keys(['nome', 'idade', 'nota'])
dict_values(['LUIZ GOMES', 39, 9.2])
```

update()

A função “**update()**” recebe um dicionário como parâmetro e insere os pares deste dicionário parâmetro no dicionário através do qual a função é chamada. Caso existam chaves coincidentes, o primeiro dicionário é atualizado com os valores do dicionário passado como parâmetro:

```
aluno_original = {"nome": "LUIZ GOMES", "idade": 39, "nota": 9.2}
aluno_update = {"peso": 100, "nota": 8.7}
aluno_original.update(aluno_update)
print(aluno_original)
```

```
{'nome': 'LUIZ GOMES', 'idade': 39, 'nota': 8.7, 'peso': 100}
```

Sets

Os sets são a última estrutura para armazenar dados que veremos. Elas não são tão comumente utilizadas, mas têm uma função.

Os sets são listas ou coleções não ordenadas de itens únicos.

CRIANDO E ACESSANDO DADOS EM SETS

Para criar um set, você pode utilizar a função “**set()**”, que transforma sequências (uma lista, por exemplo) em sets, ou você pode definir os valores do set separados entre vírgulas (mas sem formatar em pares “chave”: valor, ou você criará um dicionário):

```python
lista1 = ["Luiz Gomes", "Alfredo", "Felipe", "Alfredo", "Joana", "Carolina", "Carolina"]
set1 = set(lista1)
print(set1)
set2 = {"cachorro", "gato", "papagaio", "cachorro", "papagaio", "macaco", "galinha"}
print(set2)
```

```
{'Felipe', 'Carolina', 'Alfredo', 'Luiz Gomes', 'Joana'}
{'cachorro', 'macaco', 'gato', 'galinha', 'papagaio'}
```

Como vimos anteriormente, strings em Python também são tratadas como listas. Desta forma, também podemos usar a função “set()” para criar um set a partir de uma string:

```
set3 = set("papagaio")
print(set3)
```

```
{'g', 'a', 'i', 'o', 'p'}
```

Sets são muito usados para remover valores duplicados de listas. A forma usualmente utilizada para isso é a do exemplo abaixo:

```
lista2 = ["Luiz", "Alfredo", "Felipe", "Alfredo", "Joana", "Carolina", "Carolina"]
lista_sem_duplicatas = list(set(lista2))
print(lista_sem_duplicatas)
```

```
['Felipe', 'Carolina', 'Luiz', 'Alfredo', 'Joana']
```

Para remover valores de um set, use a função “remove()”:

```
set2 = {"cachorro", "gato", "papagaio", "cachorro", "papagaio", "macaco", "galinha"}
print(set2)
set2.remove("cachorro")
print(set2)
```

```
{'cachorro', 'macaco', 'gato', 'galinha', 'papagaio'}
{'macaco', 'gato', 'galinha', 'papagaio'}
```

Algumas funções para trabalhar com sets

Dentre as funções que também funcionam com sets, estão o “ *len()*”, que retorna a quantidade de itens no mesmo, e o uso do “in” para verificar a existência de um item no set:

```python
set2 = {"cachorro", "gato", "papagaio", "cachorro", "papagaio", "macaco", "galinha"}
print(len(set2))
print("gato" in set2)
print("elefante" in set2)
```

```
5
True
False
```

Além destas, o Python oferece algumas outras funções interessantes para o trabalho com sets. Vejamos alguns exemplos.

difference() e intersection()

As funções “ *difference()*” e “ *intersection()*” retornam, como o nome já diz, a diferença e a interseção entre dois sets, ou seja, itens que estão em um set e não estão no outro ou itens que estão nos dois sets:

```
set4 = {"Luiz Gomes", "Alfredo", "Joana", "Felipe", "Mauro"}
set5 = {"Joana", "Carolina", "Afonso", "Carlos", "Mauro"}
print(set4.difference(set5))
print(set4.intersection(set5))
```

```
{'Alfredo', 'Luiz Gomes', 'Felipe'}
{'Joana', 'Mauro'}
```

copy()

Da mesma forma que vimos para listas, para criar uma cópia de um set, deve-se utilizar a função “ *copy()*”. Criar um novo set apenas utilizando o sinal de igual faz apenas uma referência. Primeiro, vejamos como uma simples definição com o sinal de igual não resolve:

```
set4 = {"Luiz Gomes", "Alfredo", "Joana", "Felipe", "Mauro"}
set5 = {"Joana", "Carolina", "Afonso", "Carlos", "Mauro"}
set_backup = set4
print(set_backup)
set4.clear()
print(set_backup)
```

```
{'Joana', 'Alfredo', 'Felipe', 'Luiz Gomes', 'Mauro'}
set()
```

Agora, a forma que funciona:

```python
set4 = {"Luiz Gomes", "Alfredo", "Joana", "Felipe", "Mauro"}
set_backup = set4.copy()
print(set_backup)
set4.clear()
print(set_backup)
```

```
{'Joana', 'Alfredo', 'Felipe', 'Luiz Gomes', 'Mauro'}
{'Joana', 'Alfredo', 'Felipe', 'Luiz Gomes', 'Mauro'}
```

ESTRUTURAS DE CONTROLE

PROF. DR. CARLOS GUSTAVO P. MORAES

2022 - ANO DA realização

GRUPO TIRADENTES
Inspirando o futuro

Estruturas de Controle

Estruturas de Controle são utilizadas para organizar e adicionar complexidade à um programa. Elas servem diferentes finalidades, e vamos ver cada uma delas em detalhes.

If, else e elif

O “ *if*” é uma estrutura de controle que executa um determinado bloco de código baseado em uma condição. Se a condição estipulada for verdadeira, o bloco logo abaixo será executado. Este bloco de código a ser executado deve estar indentado, ou seja, com espaços em seu início, e sempre com a mesma quantidade de espaços (veja o exemplo abaixo para entender), para que o Python possa identificar onde ele começa e termina. Após a exibição da mensagem, se você digitar Felipe, uma mensagem será exibida, pois a condição estipulada na definição do bloco “ *if*” é verdadeira. Caso digite qualquer outra coisa, nada será exibido. Abaixo, considere que rodamos o programa duas vezes. Na primeira, digitamos como nome “Felipe”, e na segunda, digitamos “Maria”:

```python
nome = input("Olá, qual o seu nome? \n")
if nome == "Felipe":
    print("Olá, %s" % nome)
```

Bem, e se eu quiser imprimir outra mensagem caso a condição não seja verdadeira?
Bem, aí usamos o “ *else*”. O else é sempre executado se a condição estipulada no if não for verdadeira:

```
[7]
nome = input("Olá, qual o seu nome?\n")
if nome == "Felipe":
    print("Olá, %s" % nome)
else:
    print("Olá, visitante")
```

Por fim, e se quisermos testar mais de uma condição? Aí temos o “ *elif*”. Podemosusar o “elif” quantas vezes quisermos, para testar quantas condições forem necessárias. Se nenhuma delas for verdadeira, o bloco “*e lse*”, localizado ao final, é executado:

```
1   nome = input("Olá, qual o seu nome?\n")
2   if nome == "Felipe":
3       print("Olá, Felipe")
4   elif nome == "João":
5       print("Oi, João")
6   elif nome == "Carlos":
7       print("E aí, Carlos?")
8   else:
9       print("Olá, visitante")
10
```

Loops - For

Os loops são estruturas de controle para repetição. Imagine se eu quisesse escrever um programa que imprima todos os números de 1 a 10, ou que imprima separadamente cada item de uma determinada lista. Você poderia criar uma linha com um print para cada número, ou para cada item. Mas isso seria repetitivo, tedioso. Tem de haver uma maneira melhor de fazer isso. E têm. Para facilitar, o Python possui o loop “ *for*”. Este loop itera sobre os elementos de uma sequência, executando o código no bloco indentado logo abaixo. Pode ser uma string, uma lista. Vamos executar o código do exemplo:

```python
lista = ["Alfredo", "Mario", "José", "Carolina", "Joana", "Luiz Gomes"]
for nome in lista:
    print(nome)
```

```
Alfredo
Mario
José
Carolina
Joana
Luiz Gomes
```

Você também pode iterar sobre dicionários, mas existem diferentes formas. Como sabemos, dicionários possuem pares chave-valor. Desta forma, é possível iterar sobre apenas um ou sobre os dois. Vamos ver cada uma das formas abaixo:

```python
aluno = {"nome": "Maria", "idade": 20, "nota": 9.2}
print("Exemplo 1a - iterando sobre chaves")
for chave in aluno:
    print(chave)

print("\nExemplo 1b - iterando sobre chaves")
for chave in aluno.keys():
    print(chave)

print("\nExemplo 2 - iterando sobre valores")
for valor in aluno.values():
    print(valor)

print("\nExemplo 3 - iterando sobre ambos")
for chave, valor in aluno.items():
    print(chave + " - " + str(valor))
```

```
Exemplo 1a - iterando sobre chaves
nome
idade
nota

Exemplo 1b - iterando sobre chaves
nome
idade
nota

Exemplo 2 - iterando sobre valores
Maria
20
9.2

Exemplo 3 - iterando sobre ambos
nome - Maria
idade - 20
nota - 9.2
```

No caso da impressão dos números, podemos usar a função “range()”. Ela cria uma sequência de números inteiros, podendo receber um ou dois parâmetros. Recebendo um parâmetro, ela cria o intervalo que vai de 0, até o número passado na função, excluindo o próprio número. Caso sejam passados dois parâmetros, o intervalo começa no primeiro e termina no segundo:

```python
print("Range com 1 parâmetro")
for i in range(5):
    print(i)
print("Range com 2 parâmetros")
for i in range(3,10):
    print(i)
```

```
Range com 1 parâmetro
0
1
2
3
4
Range com 2 parâmetros
3
4
5
6
7
8
9
```

Logo, se a ideia for repetir um certo bloco de código um determinado número de vezes, basta usar o “ *range()*” com o número de repetições desejadas. E como você pode perceber através dos 2 exemplos, o nome que vem após o “ *for*” pode ser qualquer nome que siga as regras de nomenclatura de variáveis. Este nome é simplesmente o nome que se dará a cada item dentro da lista, que será diferente em cada iteração.

Dois comando importantes presentes nos loops são o “ *break*” e o “ *continue*”. Esses comandos alteram o fluxo natural de um loop. No caso do “ *break*”, ele para a execução do loop. Já no “*c ontinue*”, ele não executa mais nada naquele ciclo de iteração e passa para o próximo. Vamos dar uma olhada em um exemplo para que fique mais claro:

```python
print("Exemplo break")
for i in range(1,11):
    if i % 5 == 0:
        break
    print(i)

print("Exemplo continue")
for i in range(1,11):
    if i % 5 == 0:
        continue
    print(i)
```

```
Exemplo break
1
2
3
4
Exemplo continue
1
2
3
4
6
7
8
9
```

No primeiro exemplo, no caso do "*break*", o loop deveria imprimir todos os números de 1 a 10. Entretanto, inserimos uma condição tal que, se o valor de "i" fosse divisível por 5, ele executaria o comando "*break*". Dessa forma, quando o valor de "i" chegou a 5, o comando "b*reak*" foi executado e nada mais foi impresso.

A mesma lógica foi seguida no segundo exemplo. Entretanto, ao invés do "*break*", o comando " *continue*" foi executado, fazendo com que o loop pulasse para a próxima iteração, e não imprimindo o número 5.

GRUPO TIRADENTES
Inspirando o futuro

Loops - while

O loop while é semelhante ao for, no sentido que ele leva a repetição de um bloco de código. Entretanto, ele executa este código enquanto uma certa condição definida for verdadeira. Vejamos abaixo um exemplo semelhante ao de cima:

```
contador = 0
while contador < 5:
    print(contador)
    contador = contador + 1
```

```
0
1
2
3
4
```

Este exemplo vai imprimindo a variável contador, e no final do bloco de código, aumenta seu valor em 1. Conforme a condição, enquanto o contador for menor que 5, o bloco de código dentro dele será executado. Quando o valor chega em 5, a condição não é mais verdadeira e o bloco não é mais executado. Têm de se tomar cuidado com o loop while, pois se você definir uma condição que nunca se torne falsa, seu código fica executando sem parar. Se no exemplo acima nós não incluíssemos a última linha, que incrementa o contador em 1, o código rodaria para sempre, imprimindo zero infinitas vezes. Caso você se veja nesta situação em algum código, basta apertar Ctrl + C, e o código será interrompido.

Outra possibilidade é definir como condição para execução do loop “ *while True*”, e dentro do loop, definir alguma condição onde se execute o “ *break*”. Vejamos um exemplo um pouco mais complexo, misturando o “ *while True*” com uma estrutura “ *if / else*”:

```python
while True:
  nome = input("Digite seu nome ou sair para terminar o programa: ")
  if nome == "sair":
    break
  else:
    print("Olá, %s" % nome)
```

```
Digite seu nome ou sair para terminar o programa: LUIZ
Olá, LUIZ
Digite seu nome ou sair para terminar o programa: sair
```

FUNÇÕES

Funções são blocos de código que podem ser reutilizados quantas vezes forem necessárias, para que você não tenha que ficar escrevendo o mesmo código várias vezes. Funções podem receber ou não parâmetros, para que sejam utilizadas em seu código.

FUNÇÕES PREDEFINIDAS DO PYTHON

O Python já vem com muitas funções predefinidas. Algumas delas, já estamos usando há algum tempo, como “ *len()*” e “ *print()*”, por exemplo. Temos também algumas funções presentes em módulos, que precisam ser importados para serem utilizados.

Módulos nada mais são do que códigos e funções já definidos para serem empacotados e reutilizados em outros projetos ou outras partes de um mesmo projeto. Existem módulos já definidos pelo próprio Python, você pode criar os seus ou utilizar módulos criados por outras pessoas. Mais detalhes sobre módulos ficam para um próximo capítulo.

DEFININDO UMA FUNÇÃO

A estrutura básica para definir uma função é através da declaração “def”, depois o nome da função e parênteses. Caso a função não receba parâmetros, os parênteses ficarão vazios. Se receber, os parâmetros devem ser nomeados na definição. Primeiro, vejamos um exemplo de uma função sem parâmetros. Primeiro definimos a função, e em seguida a chamamos:

```
def print_ola_tres_vezes():
    print("Ola Python")
    print("Ola Python")
    print("Ola Python")
print_ola_tres_vezes()
```

```
Ola Python
Ola Python
Ola Python
```

GRUPO TIRADENTES
Inspirando o futuro

Funções podem ter valores retornados. Definimos o valor a ser retornado pela função através do comando “return”. O que é retornado pode ou não estar entre parênteses. Vamos ver um exemplo de uma função com um parâmetro e um valor retornado:

```python
def numero_ao_cubo(numero):
    valor_a_retornar = numero * numero * numero
    return(valor_a_retornar)
numero = numero_ao_cubo(4)
print(numero)
numero = numero_ao_cubo(5)
print(numero)
```

```
64
125
```

Uma das coisas que você pode fazer é definir um valor padrão, que será o valor daquele argumento se nenhum valor para ele for incluído na chamada da função. Para definir um argumento padrão, definimos o valor padrão na hora de definir a função, da seguinte forma:

```
def print_ola(nome="estranho"):
  print("Olá, " + nome)
print_ola("Priscilla")
print_ola()
```

```
Olá, Priscilla
Olá, estranho
```

Assim, vemos que, se providenciamos o argumento para a função, ele imprime o que é passado. Se não, ele imprime o nome padrão, definido na chamada da função. Você também pode chamar uma função e nomear os argumentos que está passando. Este é o único caso onde você pode passar os argumentos em uma ordem diferente da ordem definida na criação da função:

```
def print_infos(nome, idade):
  print("Olá, meu nome é %s e tenho %d anos" %(nome, idade))

print_infos(idade = 30, nome = "Felipe" )
```

```
Olá, meu nome é Felipe e tenho 30 anos
```

Caso você não nomeie os argumentos que está passando, eles têm de estar na mesma ordem em que foram definidos.

args e kwargs

Existe uma forma no Python de definir as funções para que recebam uma quantidade variável de argumentos, além da forma normal de definição de argumentos. Esta forma é através da definição de argumentos com um ou dois asteriscos antes de seus nomes. Por convenção, costuma-se utilizar os nomes *args e **kwargs, mas não existe uma restrição do Python para que sejam utilizados apenas estes nomes. Você pode usar o nome que quiser. Ambas as formas permitem que seja passado para a função um número qualquer de argumentos. A diferença entre eles é que com um asterisco, ou, por convenção, *args, estes argumentos não são nomeados, enquanto que se definirmos a função com o formato de dois asteriscos, utilizam-se argumentos nomeados na chamada da função. Vejamos um exemplo de cada vez:

```python
def print_tudo_2_vezes(*args):
    for parametro in args:
        print(parametro + "! " + parametro + "!")
print_tudo_2_vezes("Olá", "Luiz Gomes", "Felipe")
```

```
Olá! Olá!
Luiz Gomes! Luiz Gomes!
Felipe! Felipe!
```

GRUPO TIRADENTES
Inspirando o futuro

Como vemos, no caso do *args, você pode acessar os parâmetros passados como uma lista. No caso do **kwargs, os argumentos vem em forma de dicionário, tendo as mesmas propriedades que vimos anteriormente. Desta forma, para iterar sobre os argumentos passados, podemos fazer como no exemplo abaixo:

```python
def print_info(**kwargs):
    for parametro, valor in kwargs.items():
        print(parametro + " - " + str(valor))
print_info(nome="Luiz Gomes", idade=39, nacionalidade="Brasil")
```

```
nome - Luiz Gomes
idade - 39
nacionalidade - Brasil
```

Por fim, você pode misturar estas funcionalidades entre si e com argumentos nomeados. Vejamos apenas um exemplo para finalizar:

```python
def print_info(nome, idade, **kwargs):
    print("Nome: " + nome)
    print("Idade: " + str(idade))

    print("\nInformações adicionais:")
    for parametro, valor in kwargs.items():
        print(parametro + " - " + str(valor))


print_info(nome="Luiz Gomes", idade=39, nacionalidade="Brasil", telefone="991928161")
```

```
Nome: Luiz Gomes
Idade: 39

Informações adicionais:
nacionalidade - Brasil
telefone - 991928161
```

Como vimos, a função acima usa parâmetros nomeados, que seriam os padrões, esperados pela função, além da funcionalidade do **kwargs para receber parâmetros adicionais.

MÓDULOS

Módulos são uma forma de organizar o seu código. Basicamente, você define suas funções em um arquivo e pode reutilizá-lo facilmente em outros projetos, podendo inclusive disponibilizá-lo para a utilização de outras pessoas. Você pode definir seus próprios módulos, utilizar módulos que já vem com o Python ou instalar módulos de terceiros para utilizar em seus projetos. Vamos começar pela definição de nossos próprios módulos.

Criando e importando seu primeiro módulo

Vamos criar nosso primeiro módulo. Primeiro, salve o código abaixo em seu editor de texto com o nome “modulo_exemplo.py”:

```
[1]
def modulo_print():
  print("Oi, eu sou a primeira função dentro do módulo")
def modulo_print_com_nome(nome):
  print("Olá %s, eu sou a segunda função dentro do módulo")
```

```
modulo_exemplo.modulo_print()
modulo_print()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-2-1588eba016a8> in <cell line: 1>()
----> 1 modulo_exemplo.modulo_print()
      2 modulo_print()

NameError: name 'modulo_exemplo' is not defined
```

SEARCH STACK OVERFLOW

Você recebe um erro. O Python não sabe onde procurar por esta função. Você a definiu em outro arquivo (“modulo_exemplo.py”) e não indicou neste acima onde esta função está. E como indicamos? Fazendo um “ *import*”. O “ *import*” permite que você adicione em um arquivo código que foi definido em outro arquivo. Vamos fazer na prática para que fique claro:

PROGRAMAÇÃO ORIENTADA A OBJETOS

PROF. DR. CARLOS GUSTAVO P. MORAES

2022 - ANO DA realização

GRUPO TIRADENTES
Inspirando o futuro

PROGRAMAÇÃO ORIENTADA A OBJETOS

Programação Orientada a Objetos é uma forma de programar e organizar seu código.

DEFININDO UMA CLASSE

Para definir uma classe no Python utilizamos a declaração “class” e em seguida o nome da classe. Usamos a declaração “def __init__(self)” para definir o construtor da classe, uma função que será chamada sempre que uma instância desta classe for criada. E podemos então definir as funções, ou métodos desta classe. Vamos definir uma classe de um usuário, com um construtor e uma função qualquer:

```python
class Usuario:
    def __init__(self, nome, email):
        self.nome = nome
        self.email = email

    def diga_ola(self):
        print("Olá, meu nome é %s e meu email é %s" % (self.nome, self.email))

usuario1 = Usuario(nome="Luiz Gomes", email="luiz.gomes@souunit.com.br")

usuario1.diga_ola()
print(usuario1.nome)
```

```
Olá, meu nome é Luiz Gomes e meu email é luiz.gomes@souunit.com.br
Luiz Gomes
```

A declaração do __init__ serve para, entre outras coisas, definir as propriedades que um elemento desta classe terá. Depois, basta definir os métodos como funções normais, com a diferença que o primeiro parâmetro, na hora da definição, é o “self”. Ao chamar um método em uma instância, não há necessidade de incluir o “self”. Tanto para acessar propriedades quanto para chamar métodos, usamos a instância, um ponto e o nome do atributo ou do método.

2022 - ANO DA
realização

GRUPO TIRADENTES
Inspirando o futuro